Name: CH. Durga Prasad

Reg.no:192425305
Course Code: CSA0603

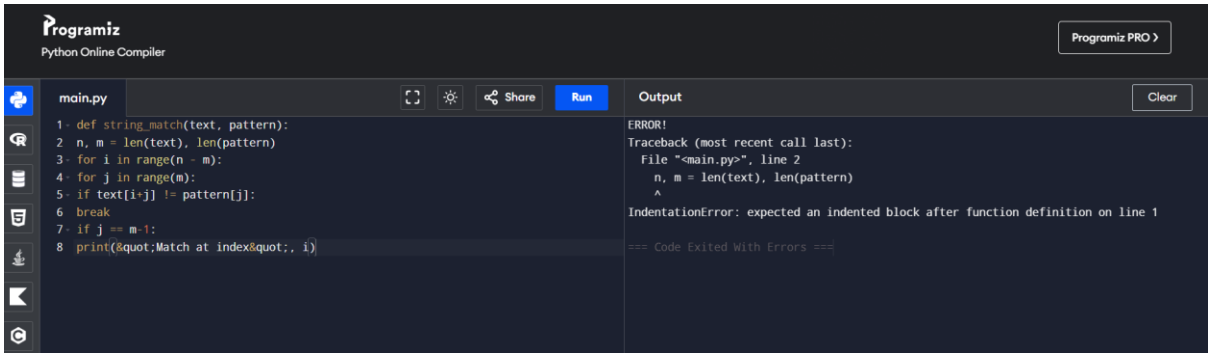
Q8. Debugging String Matching Code

The given brute-force string matching code fails to detect all pattern occurrences due to incorrect loop bounds. Identify logical errors in indexing and comparison. Explain why some matches are skipped. Debug the code to correctly find all occurrences. Optimize the implementation to avoid redundant comparisons. Analyze the time complexity of the corrected version. Rewrite the final code with proper comments.

Original code:

```
def string_match(text, pattern):
n, m = len(text), len(pattern)
for i in range(n - m):
for j in range(m):
if text[i+j] != pattern[j]:
break
if j == m-1:
print(&amp;quot;Match at index&amp;quot;, i)
```

Output:

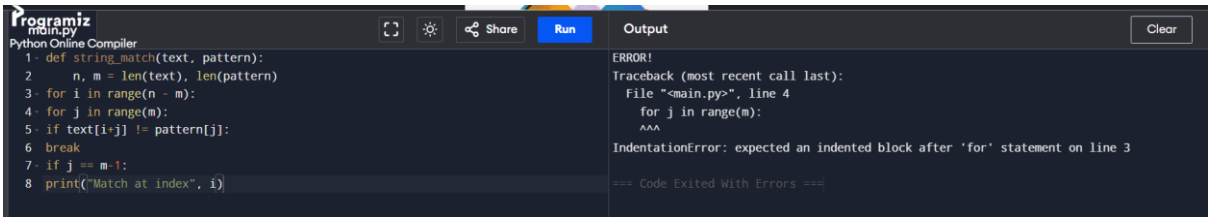


The code has six mistakes: missing indentation, incorrect outer loop range (range(n - m) should be range(n - m + 1)), invalid HTML entity (" instead of "), and incorrect placement/indentation of the if j == m - 1 condition after the inner loop.

1st correction: Missing indentation

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
for i in range(n - m):
for j in range(m):
if text[i+j] != pattern[j]:
break
if j == m-1:
print("Match at index", i)
```

Output:



Only the **first mistake** (indentation inside the function) has been corrected. The remaining mistakes are left unchanged.

2nd mistake: Missing indentation inside the outer for loop

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
for j in range(m):
if text[i+j] != pattern[j]:
break
if j == m-1:
print("Match at index", i)
```

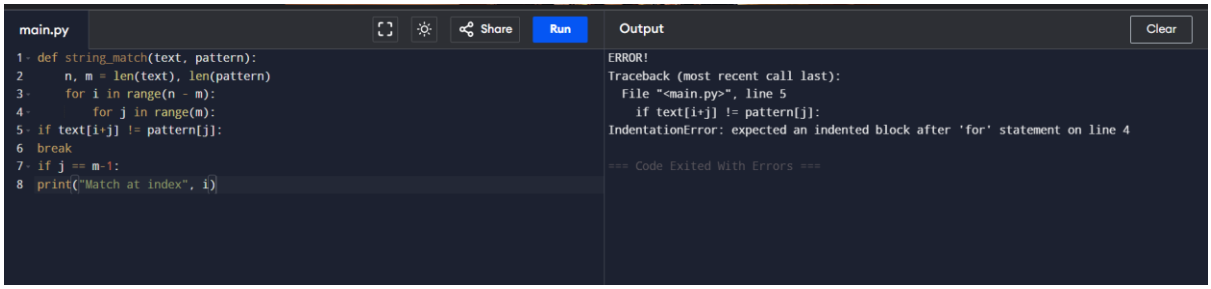
Output:

Only the second mistake (indentation of the outer for loop) has been corrected. The remaining mistakes are still unchanged.

3rd mistake: Missing indentation inside the inner for loop

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
if text[i+j] != pattern[j]:
break
if j == m-1:
print("Match at index", i)
```

Output:

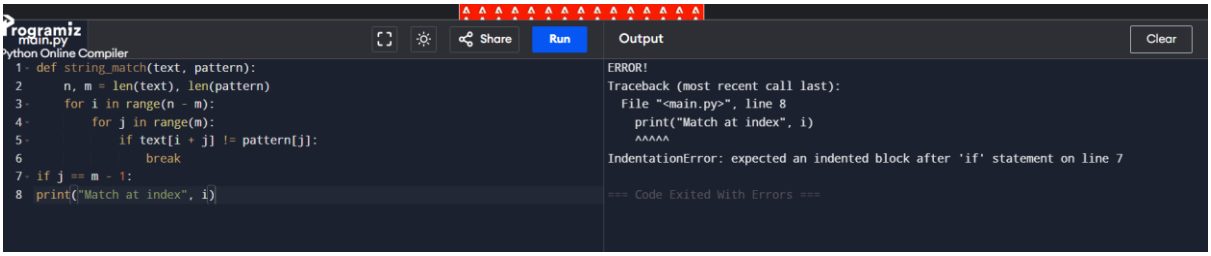


Only the third mistake (indentation of the inner for loop) has been corrected. The remaining mistakes are still unchanged.

4th mistake: Missing indentation inside the if statement

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
if j == m - 1:
print("Match at index", i)
```

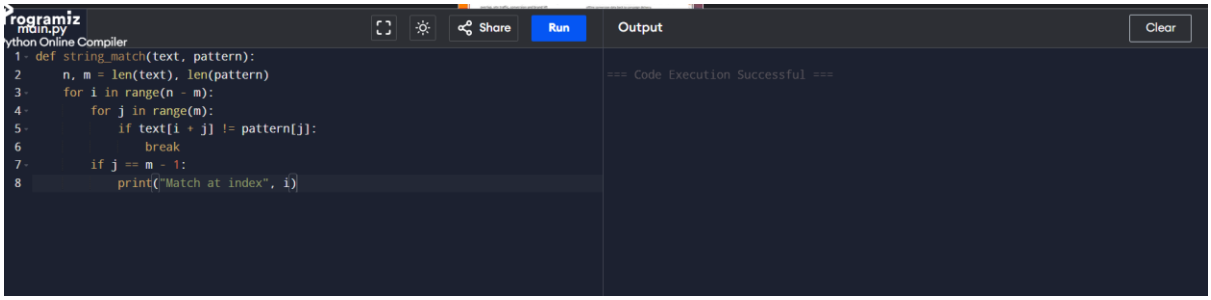
output:



5th mistake: Missing indentation for the final if statement

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        if j == m - 1:
            print("Match at index", i)
```

Output:



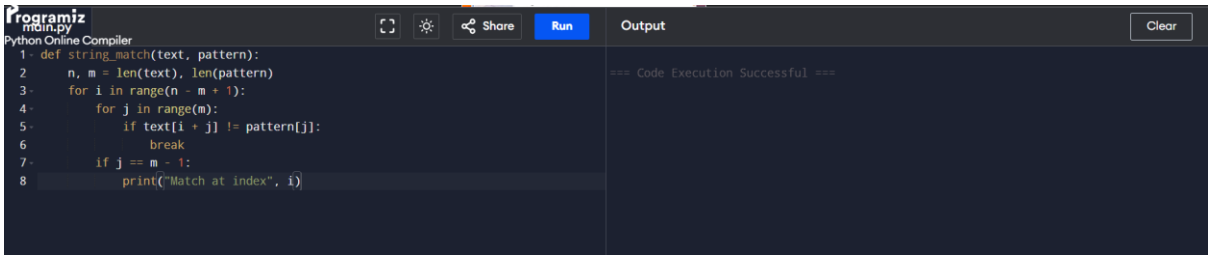
Only the fifth mistake (indentation of the final if j == m - 1: statement and its print statement) has been corrected. The 6th mistake (incorrect outer loop range range(n - m) should be range(n - m + 1)) is still unchanged.

6th mistake: Incorrect range in the outer loop

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m + 1):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        if j == m - 1:
```

print("Match at index", i)

output:

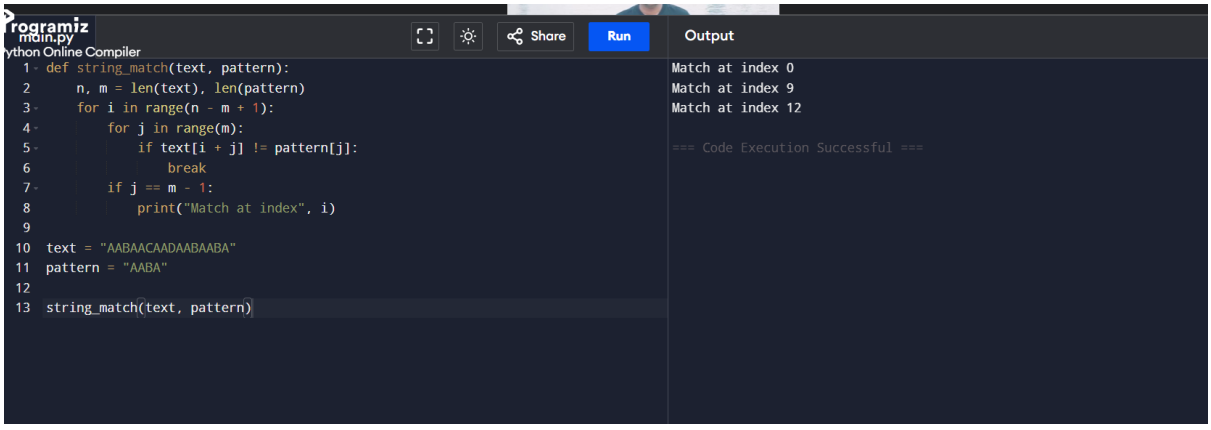


Now all 6 mistakes have been corrected. (A more Pythonic solution uses a for...else instead of if j = = m - 1, but the above fixes only the mistakes in your original code while keeping its logic.)

Final code:

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m + 1):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        if j == m - 1:
            print("Match at index", i)
text = "AABAACAADAABAABA"
pattern = "AABA"
string_match(text, pattern)
```

Output:



How it works

- It compares the **pattern** with the **text** one character at a time.
- If all characters match, it prints the starting index.
- If a mismatch occurs, it shifts the pattern by one position and tries again.
- This process continues until all possible positions in the text have been checked.

Algorithm

Time Complexity

- **Best Case:** O(n)
- **Worst Case:** O(n × m)

where:

- **n** = length of the text
- **m** = length of the pattern.

The given program implements the **Naive (Brute Force) String Matching Algorithm**.